Report 5: Chordy - A distributed hash table

Rui Zhang

October 8, 2025

1 Introduction

This report covers the implementation of a distributed hash table (DHT) based on the Chord scheme. A hash table is a data structure that implements the abstract data type of associative arrays, which can map keys to values. DHT is simply a hash table distributed across multiple nodes in a network. Each node will store the values corresponding to all the keys it is responsible for. Chord is one of the first four distributed hash table protocols, along with CAN, Tapestry, and Pastry. In Chord, nodes are arranged in a circular structure, with each node having a successor node (the next node in the ring) and a predecessor node (the previous node). Each node also has a unique ID. Chord specifies how keys are assigned to nodes and how nodes retrieve corresponding values by locating the node responsible for the key. The main theme of this work is to gain a deeper understanding of the problems caused by DHT and their solutions.

2 Main problems and solutions

node1.erl --- basic ring structure

Data structure: node(Id, Predecessor, Successor)

Core algorithm:

Consistent hashing - using key module to allocate node positions

Stabilization protocol - runs once per second to maintain the correct successor

Notify mechanism - update the predecessor when a new node joins

node2.erl --- distributed storage

Add distributed key value storage function on the basis of node1.

Data structure: node(Id, Predecessor, Successor, Store)

Core mechanism:

Key routing.

Data partitioning: Each node is responsible for the scope (Predicatesso.Id, MyId); Consistent hashing ensures load balancing.

Dynamic data migration: Automatic handover when a new node is added; Transfer some data from the old node to the new node.

node3.erl --- fault detection and recovery

Add a fault detection mechanism to enable the system to tolerate node failures.

Data structure: node(Id, Predecessor, Successor, Next, Store)

New mechanism:

Process monitoring: Ref = erlang:monitor(process, Pid) receive {'DOWN', Ref, process, _, _}

Next Node Tracking: Maintain the successor of the successor; Switch to Next immediately when the Successor fails.

Fault handling: Predecessor dies → set to nil, waiting for a new predecessor

Successor dies → adopt Next as the new successor

node4.erl --- data replication

Add a data replication mechanism to achieve a truly fault-tolerant storage system.

Data structure: node(Id, Predecessor, Successor, Next, Store, Replica)

Copy strategy:

Copy while writing: add(Key, Value) ->

Store to local Store

Copy to Successor.Replica

Return confirmation

Fault recovery: Predecessor dies → Merge Replica to Store

Successor dies → Adopt Next (Replica lost but Store retained)

Data transfer: {handover, StoreElements, ReplicaElements}

Node1 (topology)

↓

Add Storage

↓

Node2 (Function)

↓

Add monitoring

↓

Node3 (availability)

↓

Add Copy

↓

Node4 (reliability)

3 Evaluation

Base:

```
2> test:run_all_benchmarks().

========================================
Benchmark: Single Node, 4000 elements
========================================
Node 49643607 started

Adding 4000 keys...
Add time: 25.39 ms

Looking up 4000 keys...
4000 lookups in 86.43 ms (21.61 µs per lookup)
0 failed, 0 timeouts
Node 49643607 stopping (processed 4000 adds, 4000 lookups)
```

```
========================================
Benchmark: 4 Nodes, 1000 keys each
========================================
Node 741423406 started
Waiting for stabilization...
Node 560797003 started
Node 854869736 started
Node 829444918 started
Command is taking a long time, type Ctrl+G, then enter 'i' to interrupt

=== Concurrent Add Test ===

=== Node 741423406 ===

=== Node 560797003 ===

=== Node 854869736 ===

=== Node 829444918 ===
Clients: 4, Keys per client: 1000
  Predecessor: 560797003
  Predecessor: 854869736
  Predecessor: nil
  Predecessor: nil
  Successor:   560797003
  Successor:   741423406
  Successor:   560797003
  Successor:   560797003
  Store size:  0 items
  Store size:  0 items
  Store size:  0 items
  Store size:  0 items
  Statistics:  0 adds, 0 lookups
  Statistics:  0 adds, 0 lookups
  Statistics:  0 adds, 0 lookups
  Statistics:  0 adds, 0 lookups
  Range:       (560797003, 741423406]
  Range:       (854869736, 560797003]
  Range:       all keys
  Range:       all keys




Node 854869736 handed over 4 keys to 829444918
Node 829444918 received 4 keys in handover
Added 4000 keys in 1034.75 ms
Throughput: 3865.66 keys/sec

Looking up all keys...
4000 lookups in 77.93 ms (19.48 µs per lookup)
0 failed, 0 timeouts
```

We can see that as to the time of looking up, 4 nodes take less than single node based on 4000 keys.

```
========================================
Benchmark: Distributed Client Access
========================================
Node 709247922 started
Node 737038485 started
Node 726585632 started
Node 900330499 started

Test 1: All clients access N1

=== Concurrent Add Test ===
Clients: 4, Keys per client: 1000
Added 4000 keys in 27.75 ms
Throughput: 144138.95 keys/sec
Time: 31.95 ms

Test 2: Clients distributed across nodes
Time: 33.79 ms
Node 737038485 stopping (processed 60 adds, 0 lookups)
Node 709247922 stopping (processed 6511 adds, 0 lookups)
Node 900330499 stopping (processed 1298 adds, 0 lookups)
Node 726585632 stopping (processed 131 adds, 0 lookups)

========================================
Benchmark: Large Scale - 10000 elements
========================================

Test 1: Single node
Node 544767281 started
Add time: 41.98 ms
10000 lookups in 390.45 ms (39.05 µs per lookup)
0 failed, 0 timeouts

Test 2: Four nodes
Node 544767281 stopping (processed 10000 adds, 10000 lookups)
Node 893418327 started
Node 405209435 started
Node 586172407 started
Node 746074248 started
Add time: 142.44 ms
10000 lookups in 226.61 ms (22.66 µs per lookup)
0 failed, 0 timeouts
Node 893418327 stopping (processed 1470 adds, 1470 lookups)
Node 405209435 stopping (processed 5121 adds, 5121 lookups)
Node 586172407 stopping (processed 1801 adds, 1801 lookups)
Node 746074248 stopping (processed 1608 adds, 1608 lookups)
ok
```

We can see that when the elements are on a large scale, distributed computing begins to have advantages(226.61ms VS 390.45ms).

Bonus1:

```
3> N1 = node3:start(key:generate()).
<0.129.0>
4> N2 = node3:start(key:generate(), N1).
<0.132.0>
5> N3 = node3:start(key:generate(), N1).
<0.135.0>
6> N1 ! status.
Node 673314918:
  Pred: 333314988
  Succ: 943842326
  Next: 333314988
  Store: 0 items
status
7> N2 ! status.
Node 333314988:
  Pred: 943842326
  Succ: 673314918
  Next: 943842326
  Store: 0 items
status
8> N3 ! status.
Node 943842326:
  Pred: 673314918
  Succ: 333314988
  Next: 673314918
  Store: 0 items
status
9> N1 ! probe.
Probe completed in 0 microseconds, visited 3 nodes: [<0.129.0>,<0.135.0>,<0.132.0>]
probe
10> test:add([100, 200, 300], N1).
ok
11> exit(N2, kill).
Predecessor died
Successor died, adopting Next
true
12> N1 ! probe.
Probe completed in 307 microseconds, visited 2 nodes: [<0.129.0>,<0.135.0>]
probe
13> test:add([400, 500], N1).
ok
```

We can see that the ring structure is formed correctly; The Probe function is normal; Fault detection and recovery are successful; The ring self-repairs are completed, and the system continues to run. The previously added 100, 200, and 300 will be lost because node3 does not have a replication mechanism.

Bonus2:

```
2> N1 = node4:start(key:generate()).
<0.88.0>
3> N2 = node4:start(key:generate(), N1).
<0.91.0>
4> N3 = node4:start(key:generate(), N1).
<0.94.0>
5> Keys = test:keys(50).
[832062640,41586235,677182085,98000805,739620314,335480529,
 161139078,127520412,877315915,368914857,679198432,360438279,
 735915623,768127346,676473230,779202945,605569777,94059460,
 51163391,737056020,579404488,58661801,726873038,147013202,
 279996066,849646797,491391181,206916481,386298386|...]
6> test:add(Keys, N1).
ok
7> N1 ! status.
Node 601941229:
  Pred: 472184559
  Succ: 680518693
  Next: 472184559
  Store: 7 items
  Replica: 39 items
status
8> N2 ! status.
Node 680518693:
  Pred: 601941229
  Succ: 472184559
  Next: 601941229
  Store: 4 items
  Replica: 7 items
status
9> N3 ! status.
Node 472184559:
  Pred: 680518693
  Succ: 601941229
  Next: 680518693
  Store: 39 items
  Replica: 4 items
status
10> N1 ! probe.
Probe completed in 103 microseconds, visited 3 nodes: [<0.88.0>,<0.91.0>,<0.94.0>]
probe
11> exit(N2, kill).
Predecessor died, merging replica into store
Successor died, adopting Next
true
12> test:check(Keys, N1).
50 lookups in 0.31 ms (6.14 µs per lookup)
0 failed, 0 timeouts
ok
13> N1 ! probe.
Probe completed in 204 microseconds, visited 2 nodes: [<0.88.0>,<0.94.0>]
probe
```

We can see that the replication mechanism is working properly; The Probe function is normal; Both nodes have detected N2 death and performed the correct recovery operation, with zero data loss.

4 Conclusions

This assignment has been the most difficult so far, but also the most interesting. Firstly, it took some time to understand the concept of the Chord scheme. However, once this is clarified, it is necessary to implement the between function in the key module, which

requires a lot of thinking in order to correctly compare the keys. In addition, it is always necessary to think from the perspective of any node about predecessor and successor, or which node should handle which keys, which is also difficult. In addition, there were some minor difficulties encountered in setting up and testing fault handling, and it took some time to figure out what happened. The main knowledge learned is understanding the Chord scheme and general distributed hash tables.